Pensamento Computacional
INF05008

Aula 7 – Recursão, Indução e Listas

Prof. Lucas Nunes Alegre

Universidade Federal do Rio Grande do Sul

Instituto de Informática

Departamento de Informática Teórica

INSTITUTO DE INFORMÁTICA UFRGS

Tipos de Dados

- Nesta disciplina, já vimos/veremos alguns tipos de dados:
  - Básicos
    - Números
    - Strings
    - Booleanos
    - Imagens
  - Estruturados (Próximas aulas)

P. Como definir um conjunto infinito sem usar outro conjunto infinito como base?

Conjuntos Infinitos

- Conjuntos definidos por enumeração são finitos.
  - Exemplo: Semáforo = {“verde”, “vermelho”, “amarelo”}

- Conjuntos definidos através de produto cartesiano ou união serão infinitos se um dos componentes for infinito.

P. Como definir um conjunto infinito sem usar outro conjunto infinito como base?

Infinito

Infinito é o conceito de algo ilimitado, sem fim, sem fronteiras.

O símbolo do infinito foi inventado pelo matemático inglês John Wallis em 1657.



Podemos pensar em 3 tipos de infinito:

- Matemático: Por exemplo, número de pontos em uma reta, números naturais, ...
- Físico: Infinitos de espaço, tempo, dimensões (tamanho do Universo), ...
- Metafísico: O Absoluto (Deus, entidade suprema) é infinito? Outros seres podem ser infinitos?

Infinito Matemático

Grécia Antiga:

- Platão e Aristóteles rejeitavam o "infinito real" (de espaço ou tempo), mas aceitavam o "infinito potencial" (por exemplo, ser possível contar sem fim).





Infinito Matemático

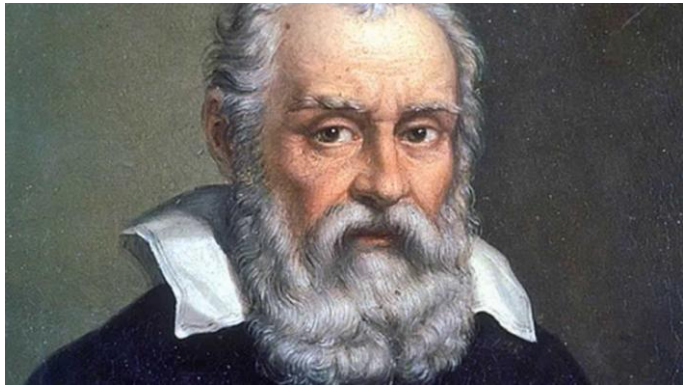


1600:

- Newton e Leibnitz descobrem os números infinitamente pequenos (infinitesimais), que dão origem ao cálculo. Foram muito criticados na época.



- Galileu expressa um suposto paradoxo sobre conjuntos:
  - Um conjunto infinito pode conter um subconjunto infinito, e os dois terem o mesmo tamanho!
  - Exemplo: O conjunto dos número naturais tem o mesmo tamanho que o conjunto dos números pares.

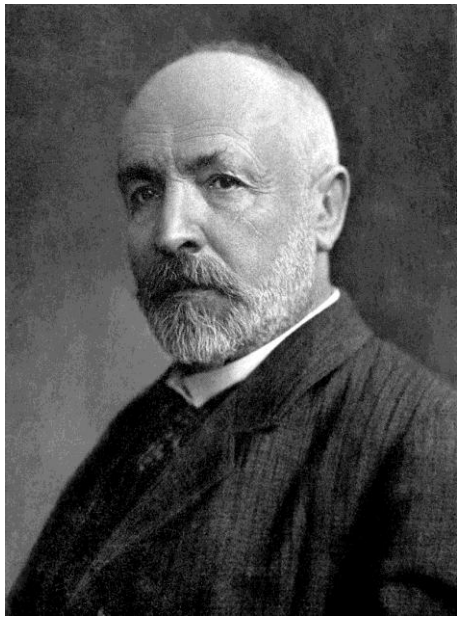


Infinitos Matemáticos

1873:

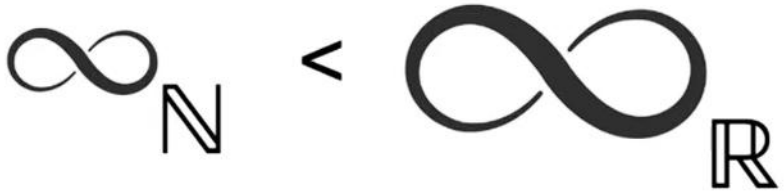
- Gerge Cantor mostra que nem todos os infinitos são iguais.
- O tamanho de "conjuntos contáveis" (números naturais) é menor que o de "conjuntos incontáveis" (números reais).

$$\infty_{\mathbb{N}} < \infty_{\mathbb{R}}$$



Infinitos Matemáticos

- Existem muitos infinitos, mas alguns são mais tratáveis que outros.



P. Por que queremos tratar infinitos?

- Para construir modelos da realidade, que potencialmente é composta de conjuntos infinitos
- Para analisar modelos da realidade
- Para construir e analisar novas realidades

Como definir conjuntos infinitos?

Queremos definir um conjunto infinito:

- De maneira finita;
- Sem usar outro conjunto infinito da definição;
- De forma a ser possível verificar propriedades dos elementos deste conjunto.

Conjunto dos Naturais

- Considere a seguinte tentativa de definição dos números naturais:
  - $\mathbb{N} = \{0, 1, 2, 3, \dots\}$

Não é uma
definição precisa!

P. Como definir estes conjuntos de forma precisa?

Indução e Recursão



Recursão

Uma definição recursiva define algo em termos de si próprio.

Uma definição recursiva tem 3 partes:

1. Base
2. Passo
3. Cláusula de exclusão

Conjunto Naturais

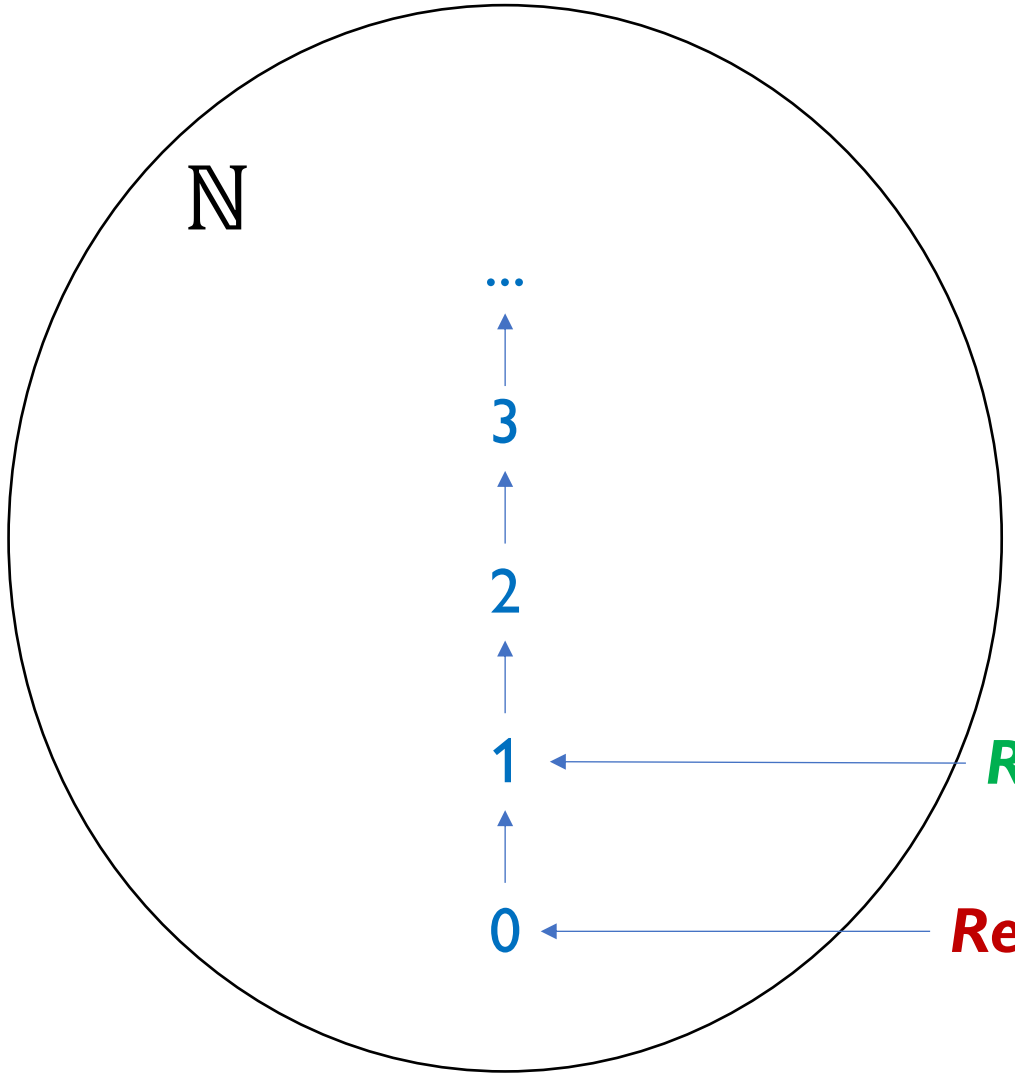
O conjunto dos números naturais pode ser definido pelas 3 regras a seguir:

R1. $0 \in \mathbb{N}$ ***(caso base)***

R2. Se $n \in \mathbb{N}$, o sucessor de n pertence a $\mathbb{N}$ ***(passo)***

R3. Somente elementos obtidos a partir das regras R1 e R3 pertencem a $\mathbb{N}$ ***(exclusão)***

Conjunto Naturais



2.1

Regra R3 (exclusão)

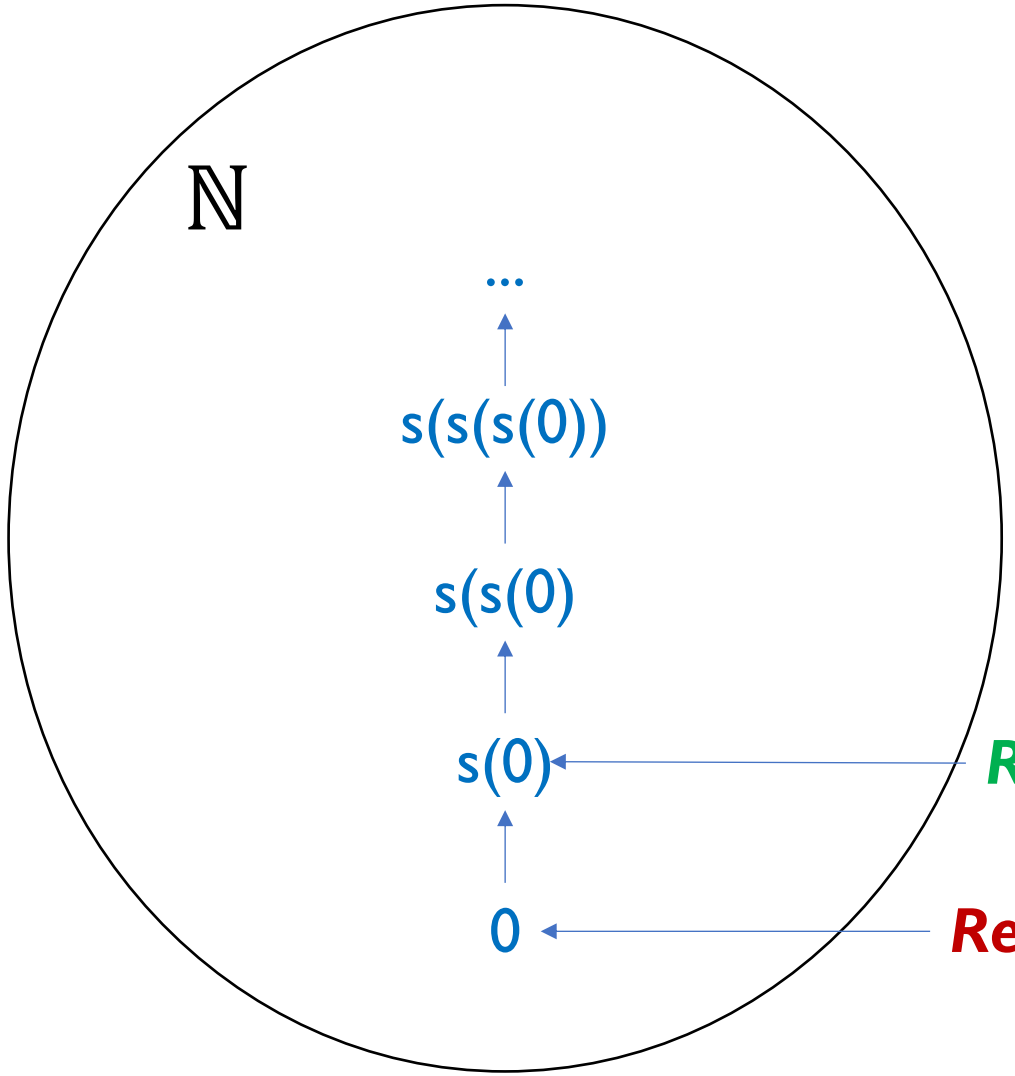
A

-4

Regra R2 (passo)

Regra R1 (caso base)

Conjunto Naturais



2.1

Regra R3 (exclusão)

A

-4

Regra R2 (passo)

Regra R1 (caso base)

Definição Recursiva

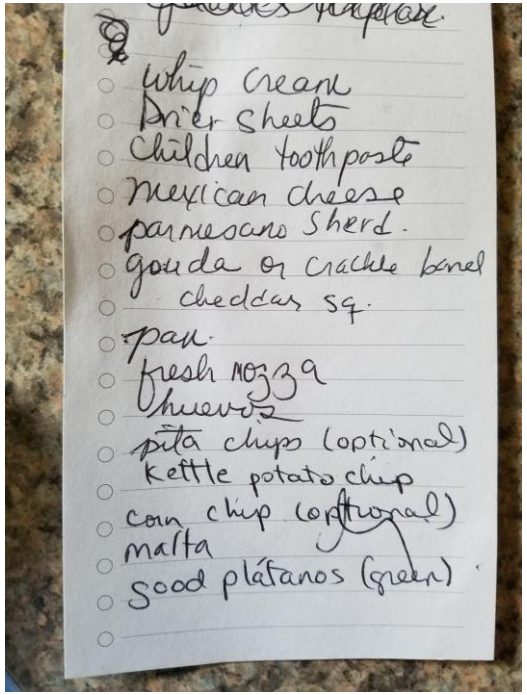
- Permite descrever conjuntos infinitos

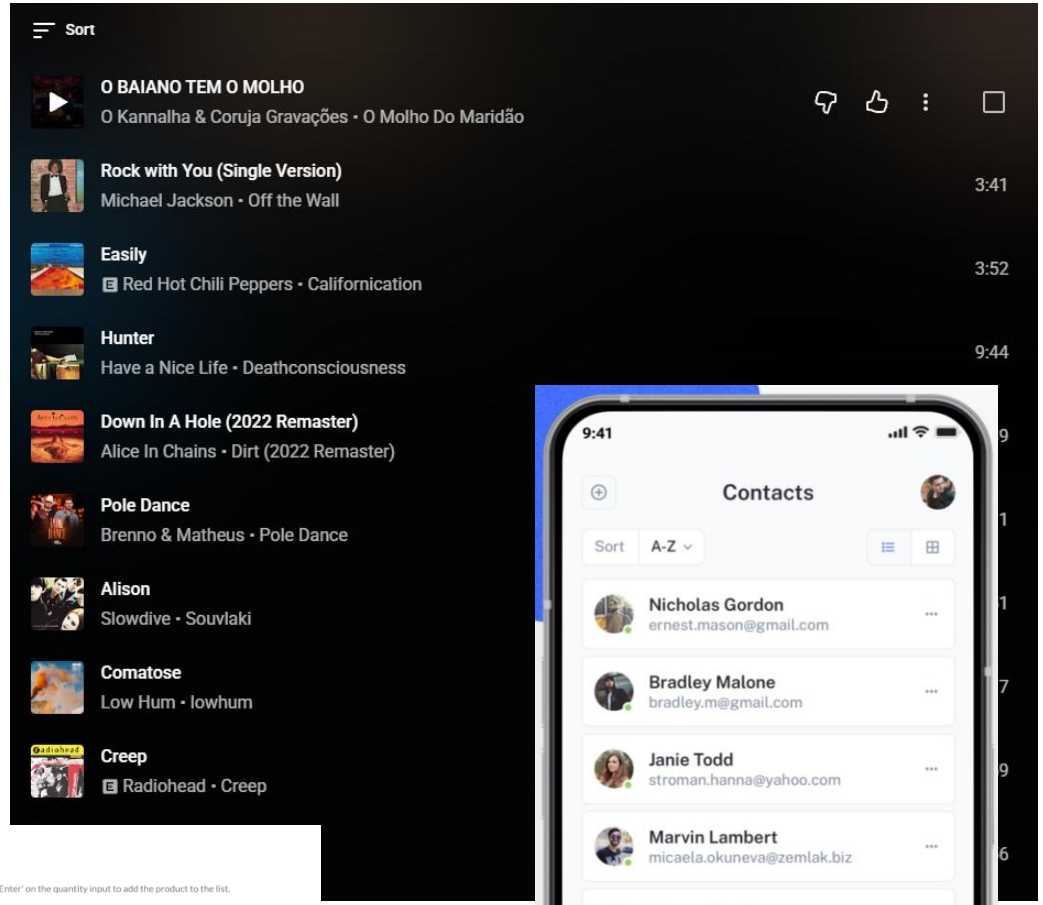
- Constrói conjuntos estruturados:

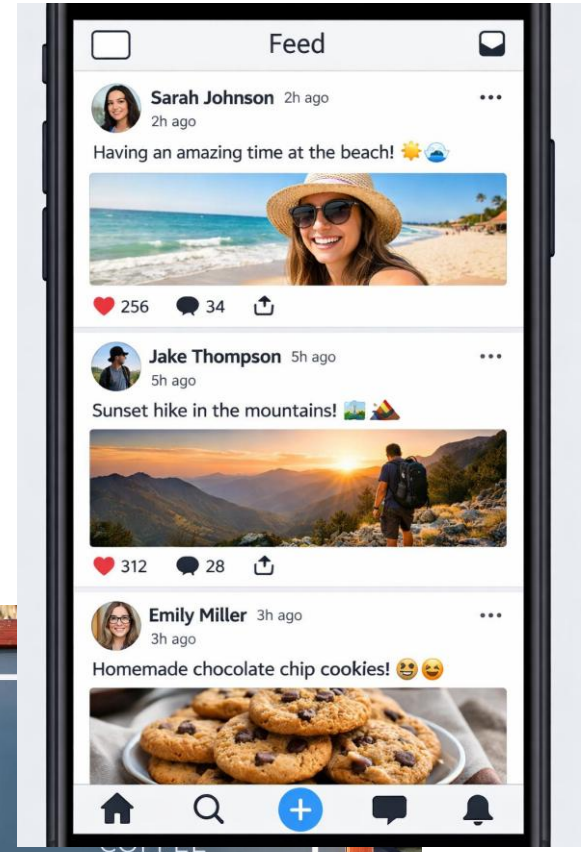
  - Existe sempre pelo menos um elemento (base)
  - Existe uma relação de ordem (parcial) entre os elementos do conjunto (passo)

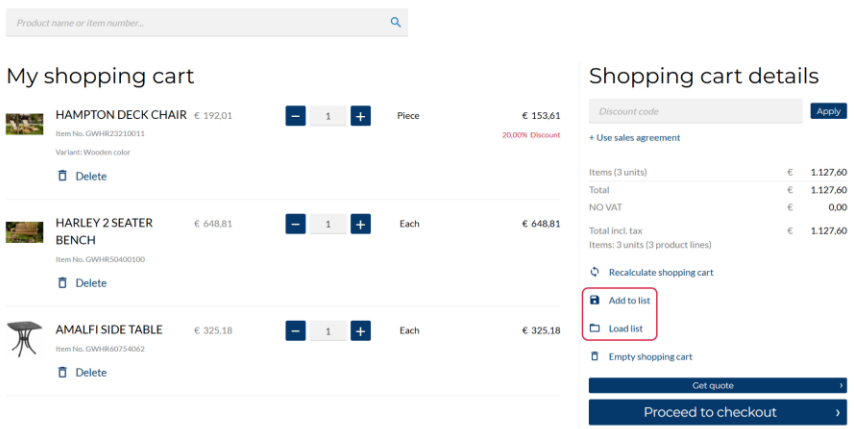
- Além disso, ela pode ser usada para:
  - Provar propriedades usando provas por indução
  - Manipular dados de tamanho arbitrário usando programas recursivos

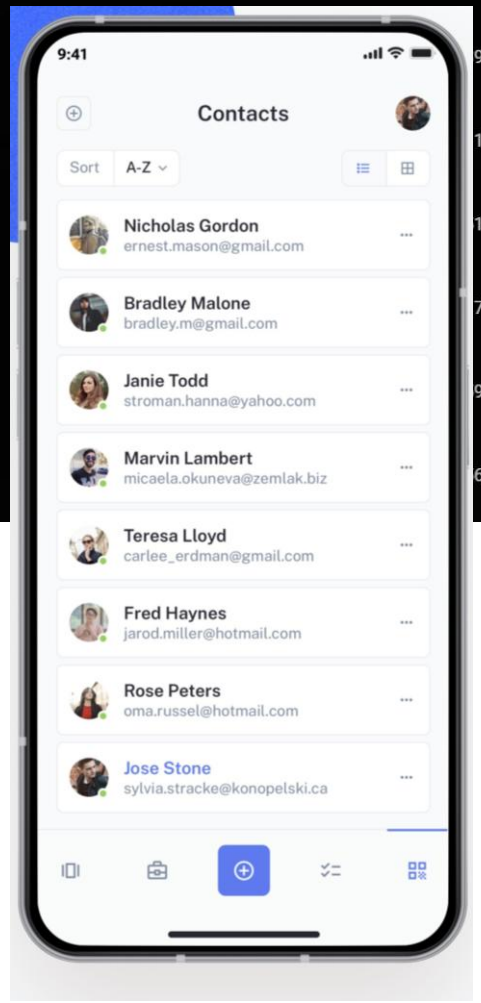
Listas

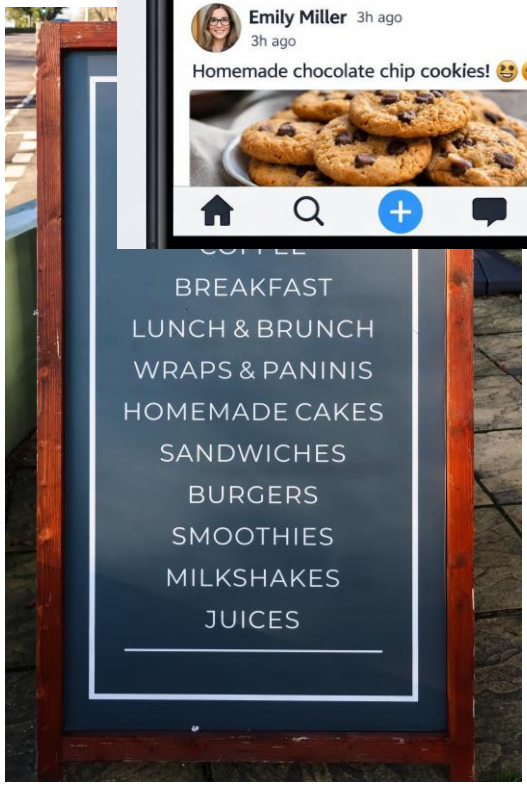

whip cream
Brier sheets
Children toothpaste
mexican cheese
parmesano sherd.
gouda or cracker barrel
cheddar sq.
pan.
fresh mozza
chuevos
pita chips (optional)
kettle potato chip
corn chip (optional)
malta
sood plátanos (green)


Sort
O BAIANO TEM O MOLHO
O Kannalha & Coruja Gravações • O Molho Do Maridão
Rock with You (Single Version)
Michael Jackson • Off the Wall
3:41
Easily
Red Hot Chili Peppers • Californication
3:52
Hunter
Have a Nice Life • Deathconsciousness
9:44
Down In A Hole (2022 Remaster)
Alice In Chains • Dirt (2022 Remaster)
Pole Dance
Brenno & Matheus • Pole Dance
Alison
Slowdive • Souvlaki
Comatose
Low Hum • lowhum
Creep
Radiohead • Creep


Feed
Sarah Johnson 2h ago
2h ago
Having an amazing time at the beach!
256
34
Jake Thompson 5h ago
5h ago
Sunset hike in the mountains!
312
28
Emily Miller 3h ago
3h ago
Homemade chocolate chip cookies!


Shopping cart
Enter an item number and press 'Enter' to load the product information and variants. Tab to select variants and set quantity. Press 'Enter' on the quantity input to add the product to the list.
Product name or item number...
My shopping cart
HAMPTON DECK CHAIR € 192,01
Item No. GWHR23210011
Variant: Wooden color
Delete
Piece
€ 153,61
20,00% Discount
HARLEY 2 SEATER BENCH € 648,81
Item No. GWHR50400100
Delete
Each
€ 648,81
AMALFI SIDE TABLE € 325,18
Item No. GWHR60754062
Delete
Each
€ 325,18
Shopping cart details
Discount code
Apply
+ Use sales agreement
Items (3 units) € 1.127,60
Total € 1.127,60
NO VAT € 0,00
Total incl. tax € 1.127,60
Items: 3 units (3 product lines)
Recalculate shopping cart
Add to list
Load list
Empty shopping cart
Get quote
Proceed to checkout


9:41
Contacts
Sort
A-Z
Nicholas Gordon
ernest.mason@gmail.com
Bradley Malone
bradley.m@gmail.com
Janie Todd
stroman.hanna@yahoo.com
Marvin Lambert
micaela.okuneva@zemlak.biz
Teresa Lloyd
carlee_erdman@gmail.com
Fred Haynes
jarod.miller@hotmail.com
Rose Peters
oma.russel@hotmail.com
Jose Stone
sylvia.stracke@konopelski.ca


BREAKFAST
LUNCH & BRUNCH
WRAPS & PANINIS
HOMEMADE CAKES
SANDWICHES
BURGERS
SMOOTHIES
MILKSHAKES
JUICES

Lista

- Uma lista é uma estrutura fundamental na computação usada para agrupar elementos.

- Possui tamanho indeterminado;
- Não se pode acessar qualquer elemento diretamente;
  - É necessário percorrer à partir do primeiro elemento.

- Ordem: Os elementos têm ordem definida (primeiro, segundo, último).
- Tipagem Homogênea: Todos os elementos de uma mesma lista são do mesmo tipo.
- Dado Anônimo: A lista em si não diz "o que" ela representa (ex: idades? preços?). Você, como programador(a), é quem dá o contexto.

Definindo uma Lista

- Uma lista é um tipo de dado recursivo
- Como visto anteriormente, toda definição recursiva necessita de um caso base e um passo indutivo

P. Qual o caso base de uma lista?

- A lista vazia

P. Como criamos uma lista a partir de uma lista vazia?

- Inserindo um elemento na lista vazia

P. Como criamos novas listas a partir de uma lista?

- Inserindo (ou removendo) elementos

Definindo uma Lista

- Vamos tentar definir uma Lista de Números!

```
data NumList:
    |
end
```

- Primeiro, precisamos de um caso base, a lista vazia
  - Vamos chamá-la de nl-empty

Definindo uma Lista

- Vamos tentar definir uma Lista de Números!

```
data NumList:
    | nl-empty  # lista vazia

end
```

- Primeiro, precisamos de um caso base, a lista vazia
  - Vamos chamá-la de nl-empty
  - Poderíamos escolher outro nome, e.g., *vazia*

Definindo uma Lista

- Vamos tentar definir uma Lista de Números!

```
data NumList:
    | nl-empty  # lista vazia

end
```

- Agora, precisamos definir como construímos uma lista a partir de uma lista existente
- Podemos "linkar" um elemento a uma lista, inserindo no início da lista

Definindo uma Lista

- Vamos tentar definir uma Lista de Números!

```
data NumList:
    | nl-empty  # lista vazia
    | nl-link(first :: _____, rest :: ______)
    # first é o primeiro número da lista, rest é o restante da lista
end
```

- Agora, precisamos definir como construímos uma lista a partir de uma lista existente
- Podemos "linkar" um elemento a uma lista, inserindo no início da lista
- Qual o tipo de first?
- Qual o tipo de rest?

Definindo uma Lista

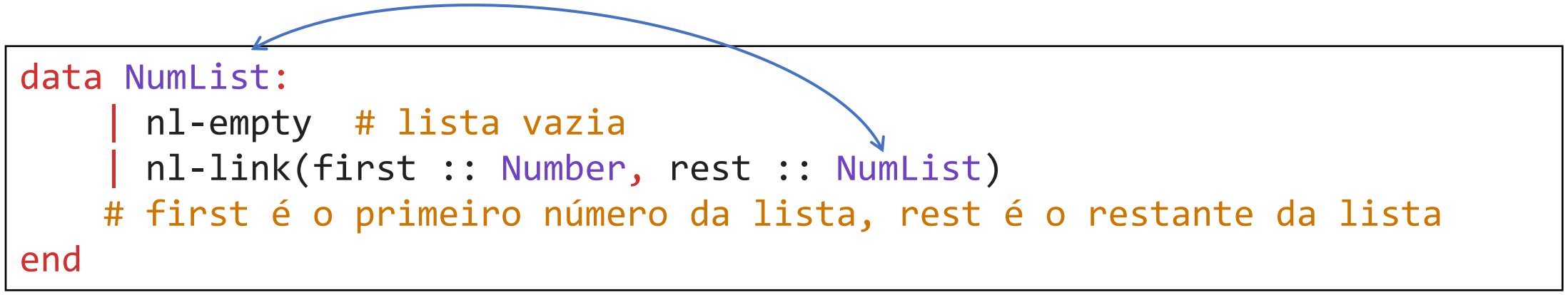
- Vamos tentar definir uma Lista de Números!

```
data NumList:
    | nl-empty  # lista vazia
    | nl-link(first :: Number, rest :: NumList)
   # first é o primeiro número da lista, rest é o restante da lista
end
```

- Agora, precisamos definir como construímos uma lista a partir de uma lista existente
- Podemos "linkar" um elemento a uma lista, inserindo no início da lista
- Qual o tipo de first? Number
- Qual o tipo de rest? NumList

Definindo uma Lista

- Temos uma definição recursiva!



```
data NumList:
    | nl-empty  # lista vazia
    | nl-link(first :: Number, rest :: NumList)
    # first é o primeiro número da lista, rest é o restante da lista
end
```

Exemplos:

```
nl-empty
nl-link(3, nl-empty)
nl-link(7, nl-link(3, nl-empty))
nl-link(2, nl-link(7, nl-link(3, nl-empty)))
```

Definindo uma Lista

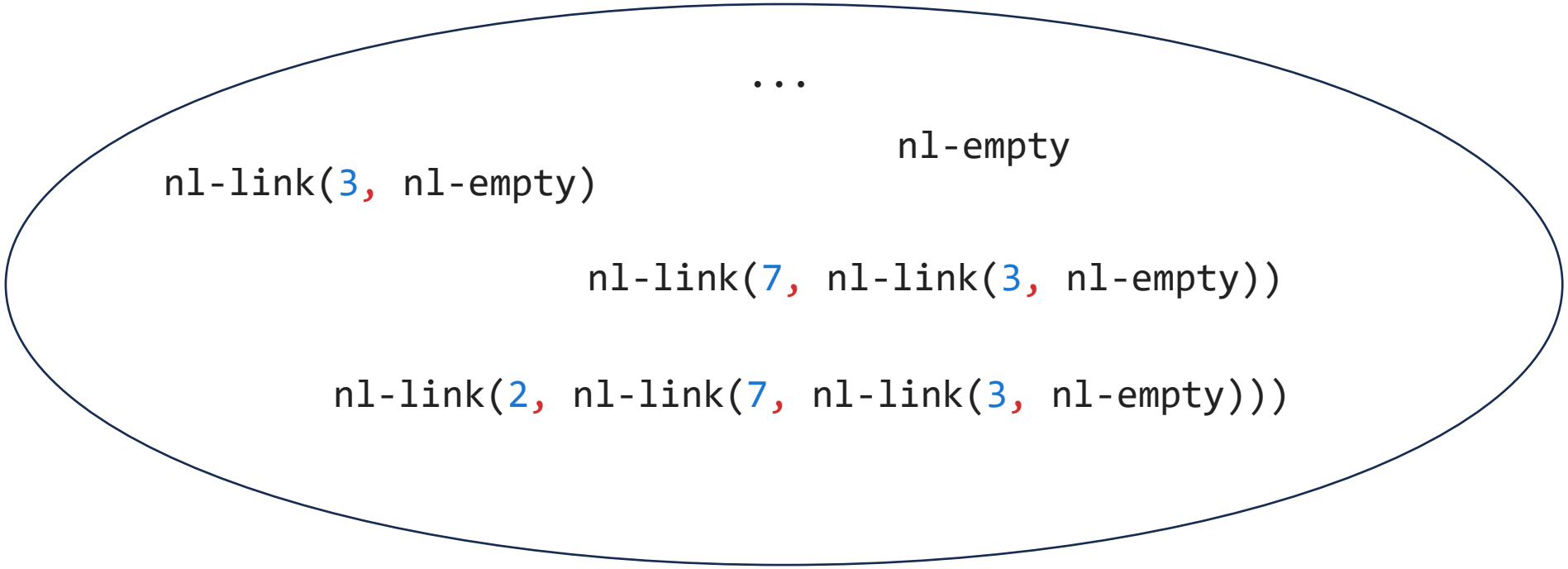
Exemplo:

```
nl-link(1,
  nl-link(2,
    nl-link(3,
      nl-link(4,
        nl-link(5,
          nl-link(6,
            nl-link(7,
              nl-link(8,
                nl-empty)))))))
```

- Acima, o argumento *rest* é sempre uma NumList válida.
- Definições de dados que contém campos do próprio tipo são chamadas de *dados recursivos*.
- Definições de dados recursivos são poderosas porque nos permitem criar dados de *tamanho arbitrário*.

Conjunto NumList

```
data NumList:
    | nl-empty  # lista vazia
    | nl-link(first :: Number, rest :: NumList)
    # first é o primeiro número da lista, rest é o restante da lista
end
```



Conjunto Naturais

O conjunto dos números naturais pode ser definido pelas 3 regras a seguir:

R1. $0 \in \mathbb{N}$ ***(caso base)***

R2. Se $n \in \mathbb{N}$, o sucessor de n pertence a $\mathbb{N}$ ***(passo)***

R3. Somente elementos obtidos a partir das regras R1 e R2 pertencem a $\mathbb{N}$ ***(exclusão)***

Conjunto NumList

O conjunto das Listas de Números pode ser definido pelas 3 regras a seguir:

R1. nl-empty ∈ NumList *(caso base)*

R2. Se n ∈ Number e l ∈ NumList, então nl-link(n, l) ∈ NumList *(passo)*

R3. Somente elementos obtidos a partir das regras R1 e R2 pertencem a $\mathbb{N}$ *(exclusão)*

```
data NumList:
    | nl-empty  # lista vazia
    | nl-link(first :: Number, rest :: NumList)
    # first é o primeiro número da lista, rest é o restante da lista
end
```

Prova por Indução

Teorema. Qualquer lista de números criadas estritamente pelas regras nl-empty e link tem um tamanho finito.

Hipótese de Indução.
Vamos assumir que temos uma lista l cujo tamanho é finito $k \in \mathbb{N}$.

Caso Base. A lista nl-empty tem tamanho $k = 0$, e portanto é finita.

Passo de Indução.

Prova por Indução

Passo de Indução. Vamos assumir que temos uma lista l cujo tamanho é finito $k \in \mathbb{N}$.

A partir dessa lista, se adicionarmos um novo número c através de nl-link, ela pode se tornar infinita de repente?

- Temos que $tamanho(l) = k$
- Se gerarmos uma nova lista adicionando c a lista l seu tamanho será:
  - $tamanho(\text{nl–link(c, l)}) = 1 + tamanho(l) = 1 + k$
- $1 + k$ também é finito. Logo, toda lista sempre tem um tamanho que é um número natural.

Mas como podemos construir um algoritmo para calcular o tamanho de uma lista?

Funções para Processar Dados Recursivos

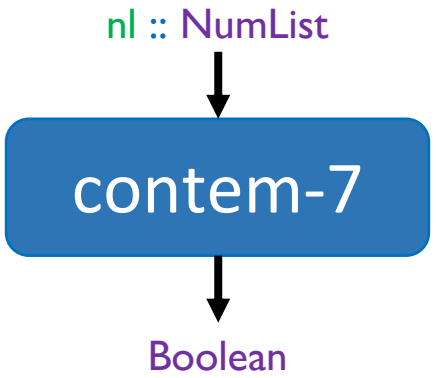
Problema

Encontrar o Número 7

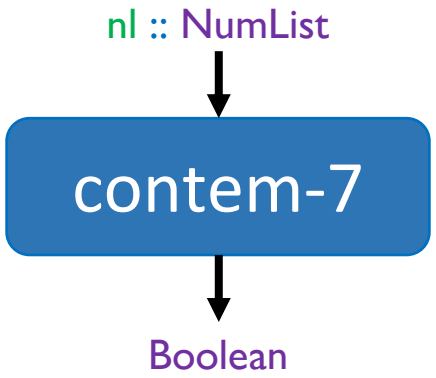
Obj: Dada uma lista de números, indicar se o número 7 está presente na lista.

Entrada: Uma lista de números (NumList).

Saída: *true* caso o número 7 esteja na lista, *false* caso contrário.

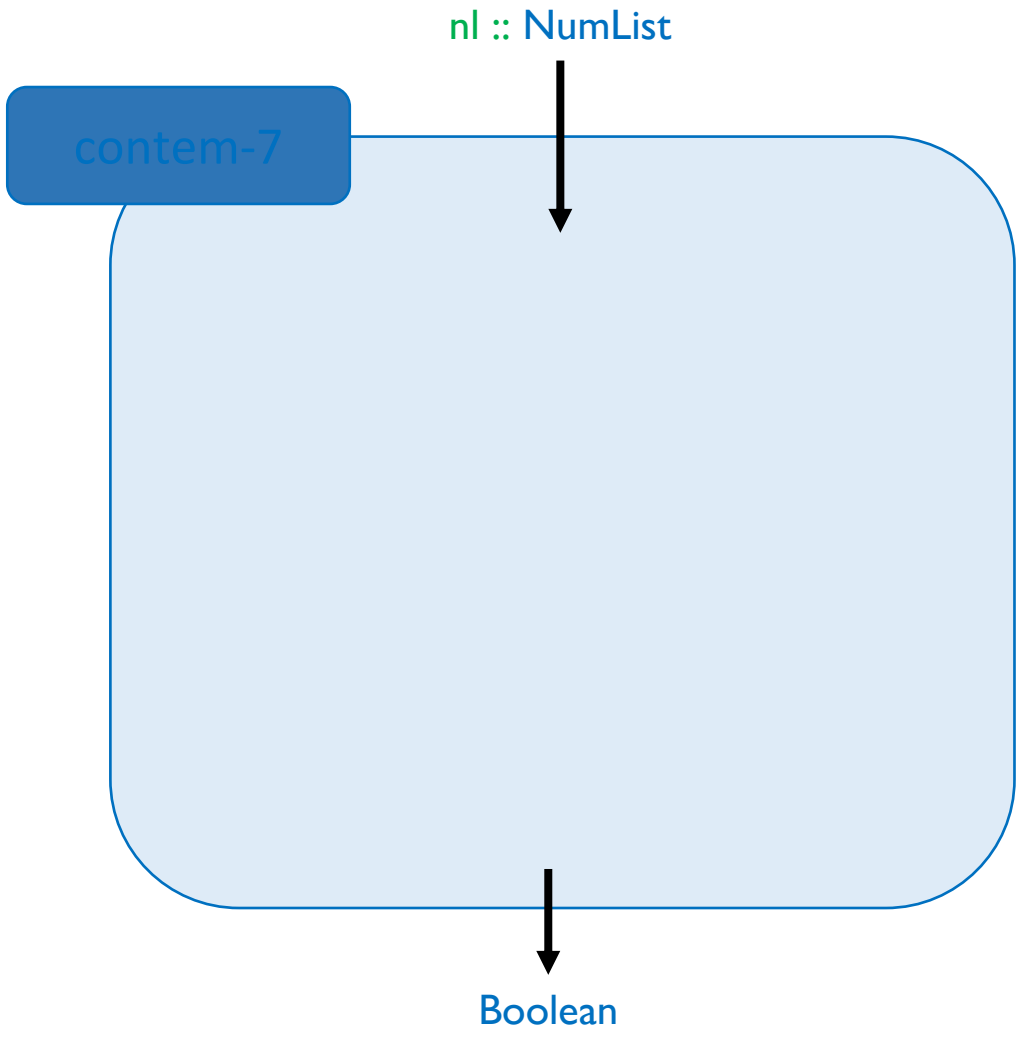


Restrições



- As operações básicas que pode-se realizar são:
  - Olhar o primeiro elemento da lista — first
  - Retirar o primeiro elemento da lista — rest
  - Verificar se um número é igual a 7 — == 7
  - Condicionais — ask:

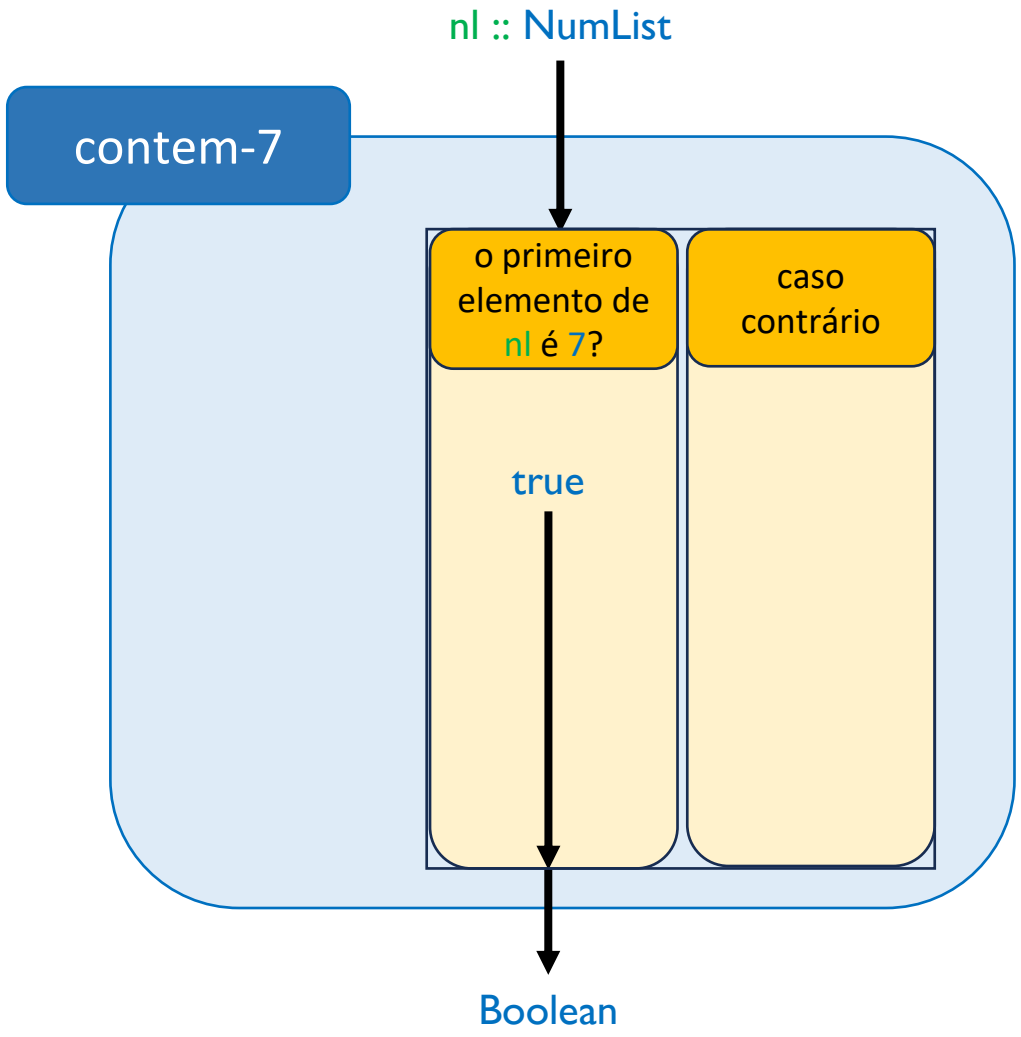
Contem 7




```
fun contem-7(nl :: NumList) -> Boolean:
  doc: "Dado uma lista de números, retorna true se
ela contém o número 7, e false caso contrário."


where:
  contem-7(nl-empty) is false
  contem-7(nl-link(1, nl-link(7, nl-empty))) is true
  contem-7(nl-link(1, nl-link(2, nl-link(3, nl-
empty)))) is false
end
```

Contem 7

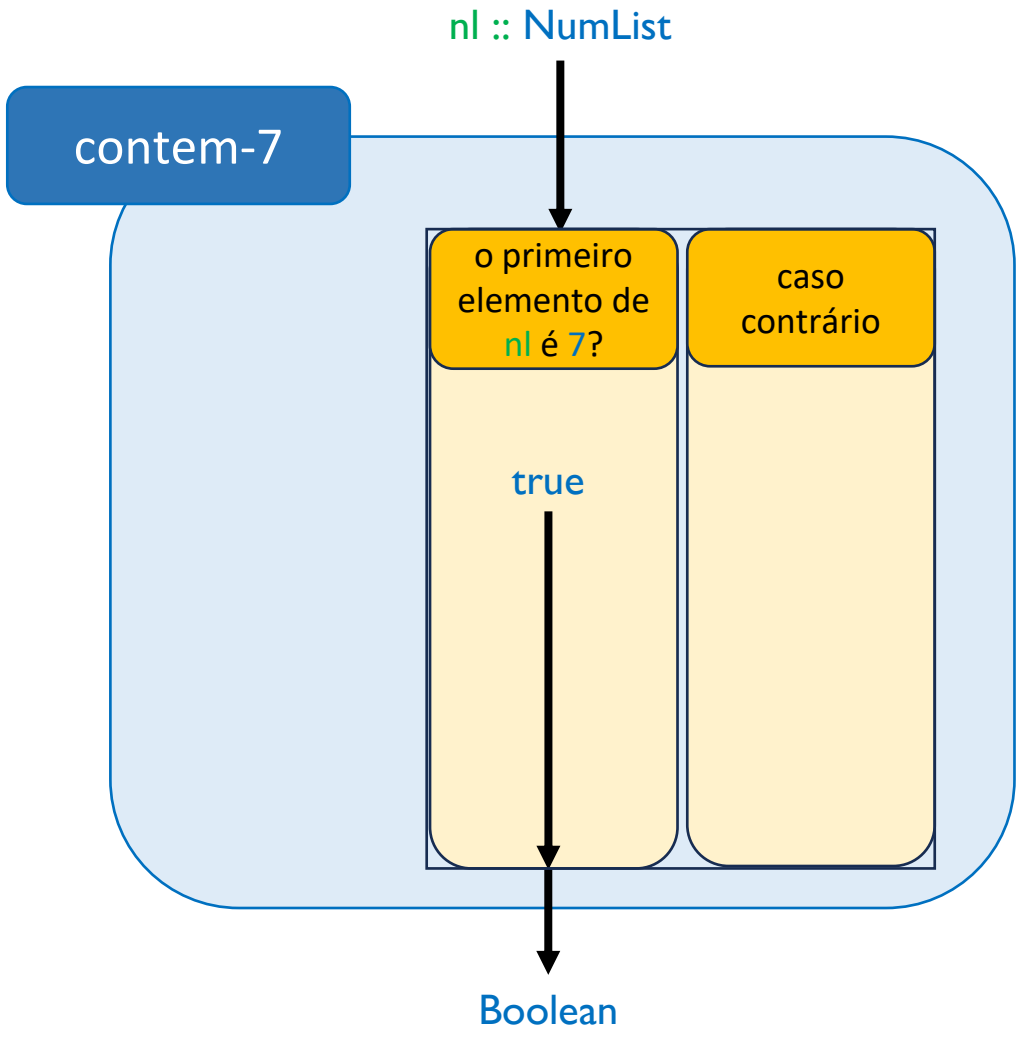



```
fun contem-7(nl :: NumList) -> Boolean:
  doc: "Dado uma lista de números, retorna true se
ela contém o número 7, e false caso contrário."

   # Se o primeiro número da lista for 7, então a
retorna true
   if nl.first == 7:
      true
   else:
       ???
   end

where:
  contem-7(nl-empty) is false
  contem-7(nl-link(1, nl-link(7, nl-empty))) is true
  contem-7(nl-link(1, nl-link(2, nl-link(3, nl-
empty)))) is false
end
```

Contem 7



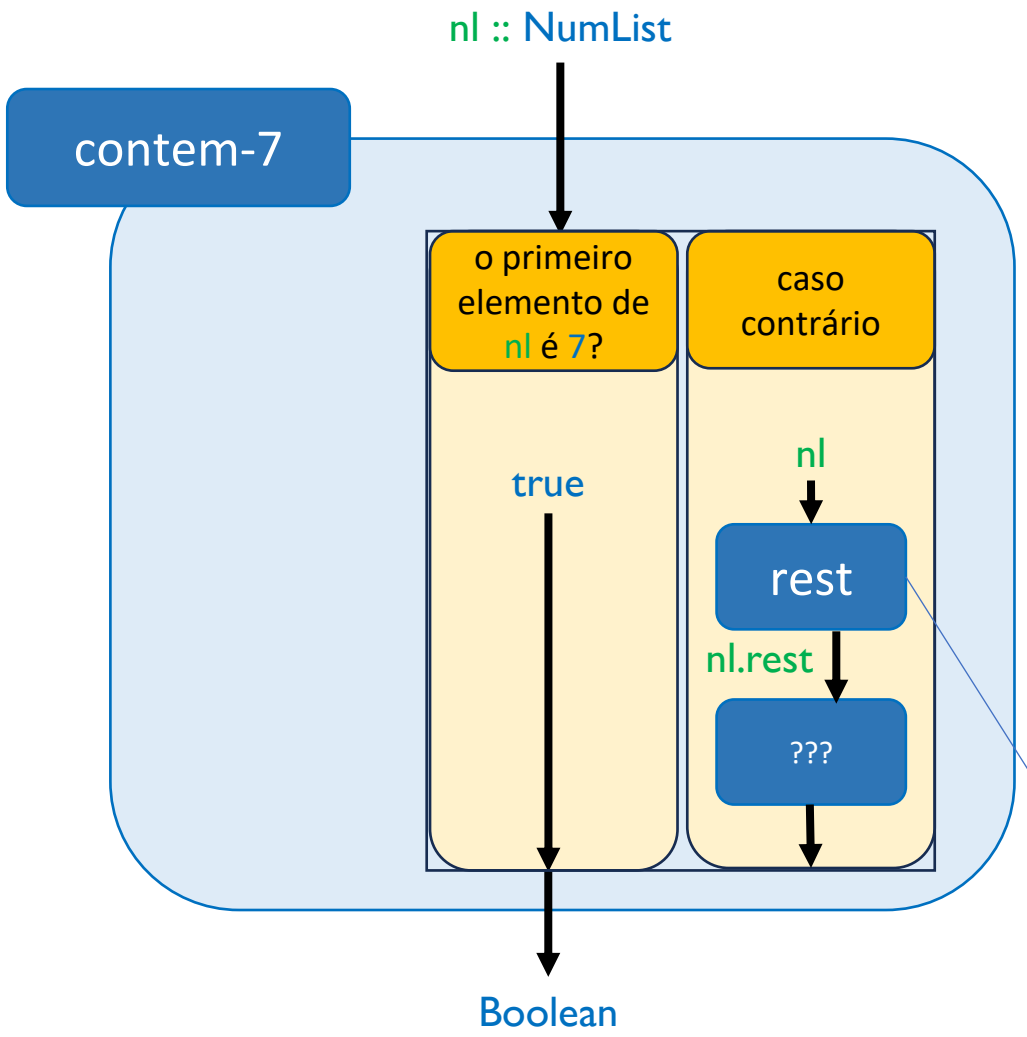

```
fun contem-7(nl :: NumList) -> Boolean:
  doc: "Dado uma lista de números, retorna true se
ela contém o número 7, e false caso contrário."

   # Se o primeiro número da lista for 7, então a
retorna true
   if nl.first == 7:
      true
   else:
       ???
   end

where:
  contem-7(nl-empty) is false
  contem-7(nl-link(1, nl-link(7, nl-empty))) is true
  contem-7(nl-link(1, nl-link(2, nl-link(3, nl-
empty)))) is false
end
```

O primeiro do resto da lista é 7?

Contem 7



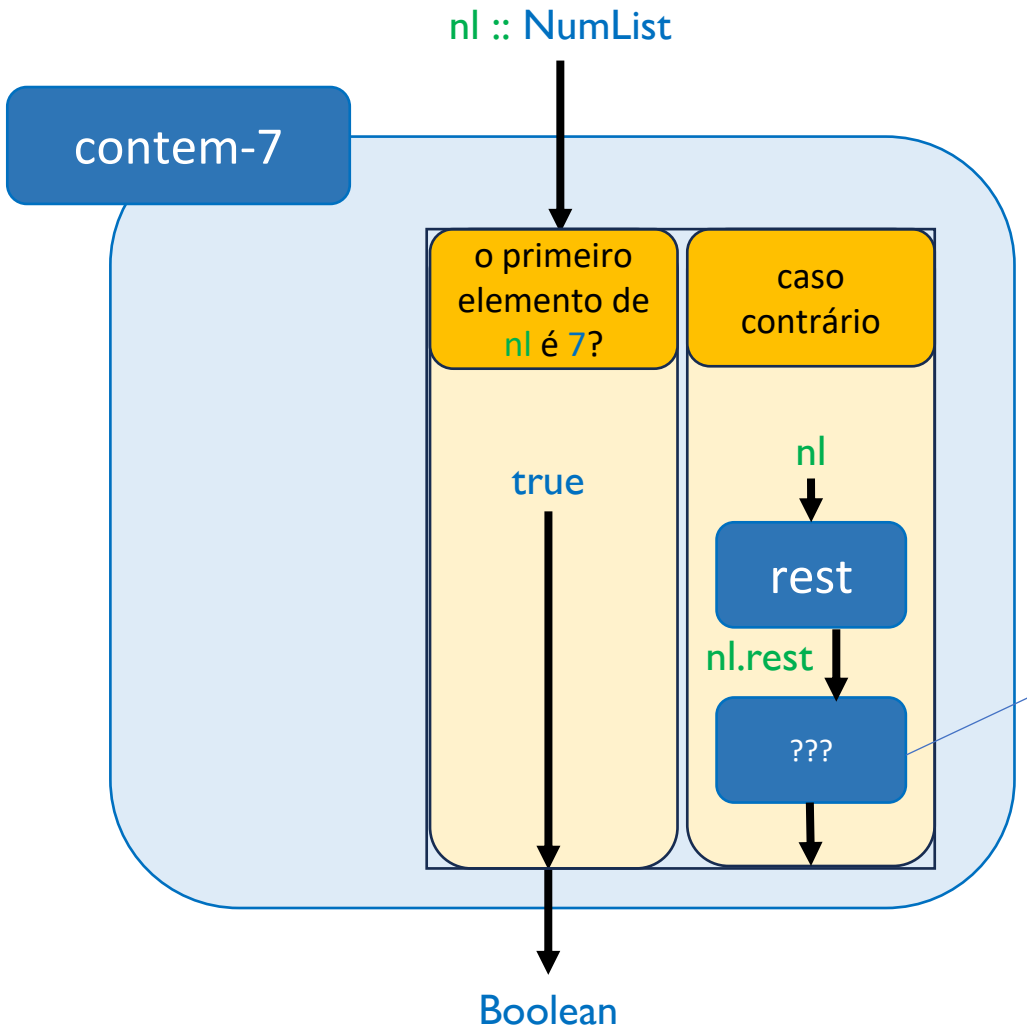

```
fun contem-7(nl :: NumList) -> Boolean:
  doc: "Dado uma lista de números, retorna true se
ela contém o número 7, e false caso contrário."

   # Se o primeiro número da lista for 7, então a
retorna true
   if nl.first == 7:
      true
   # Se não, verificar se o resto da lista contem o
número 7
   else:
       ???(nl.rest)
   end

where:
  contem-7(nl-empty) is false
  contem-7(nl-link(1, nl-link(7, nl-empty))) is true
  contem-7(nl-link(1, nl-link(2, nl-link(3, nl-
empty)))) is false
end
```

Remover primeiro número da lista

Contem 7

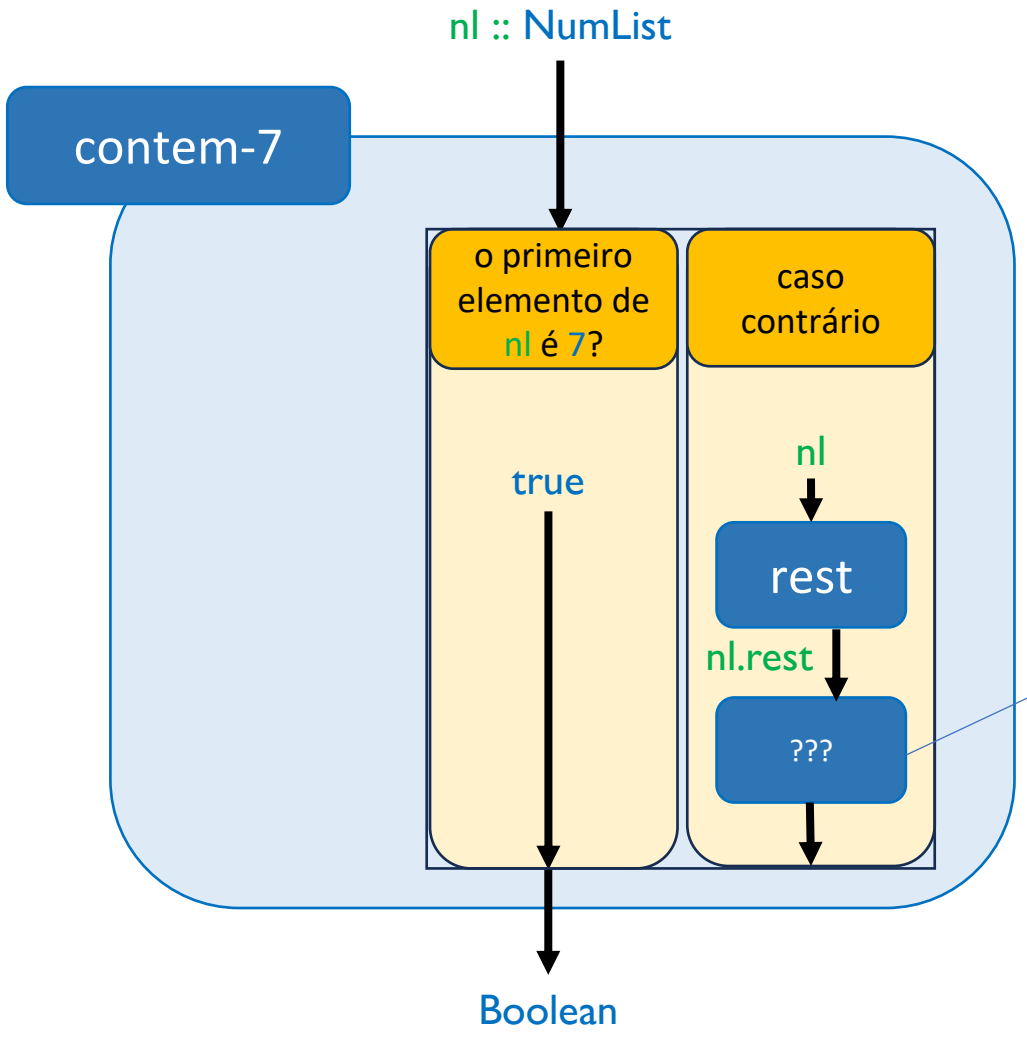



```
fun contem-7(nl :: NumList) -> Boolean:
  doc: "Dado uma lista de números, retorna true se
ela contém o número 7, e false caso contrário."

   # Se o primeiro número da lista for 7, então a
retorna true
   if nl.first == 7:
      true
   # Se não, verificar se o resto da lista contem o
número 7
   else:
       if nl.rest.first == 7:
         true
       else:
         ...
   end
where:
  contem-7(nl-empty) is false
  contem-7(nl-link(1, nl-link(7, nl-empty))) is true
  contem-7(nl-link(1, nl-link(2, nl-link(3, nl-
empty)))) is false
end
```

O primeiro do resto da lista é 7?

Contem 7



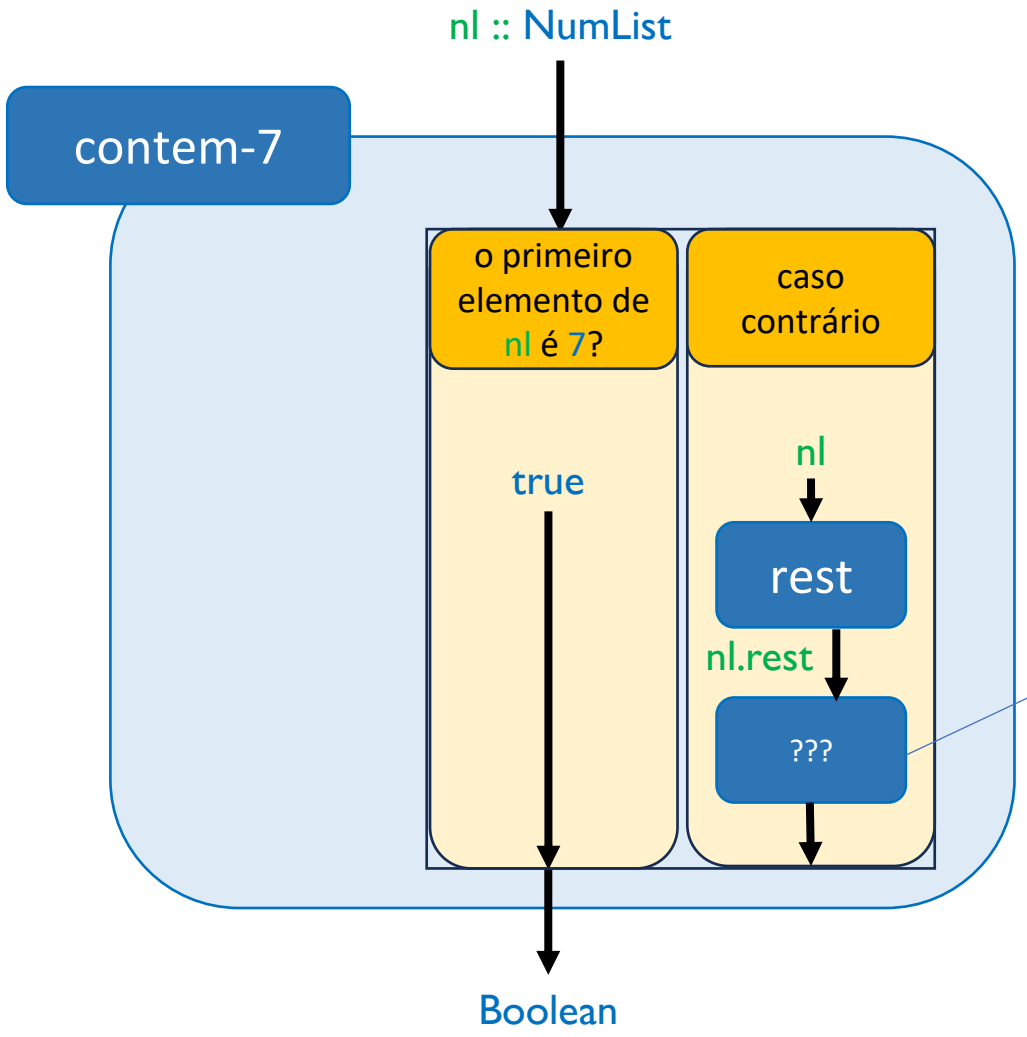

```
fun contem-7(nl :: NumList) -> Boolean:
  doc: "Dado uma lista de números, retorna true se
ela contém o número 7, e false caso contrário."

   # Se o primeiro número da lista for 7, então a
retorna true
   if nl.first == 7:
      true
   # Se não, verificar se o resto da lista contem o
número 7
   else:
       if nl.rest.first == 7:
         true
       else:
         if nl.rest.rest.first == 7:
            true
          else:
              ...
   end

where:
  contem-7(nl-empty) is false
  contem-7(nl-link(1, nl-link(7, nl-empty))) is true
  contem-7(nl-link(1, nl-link(2, nl-link(3, nl-
empty)))) is false
end
```

O primeiro do resto do resto da lista é 7?

Contem 7



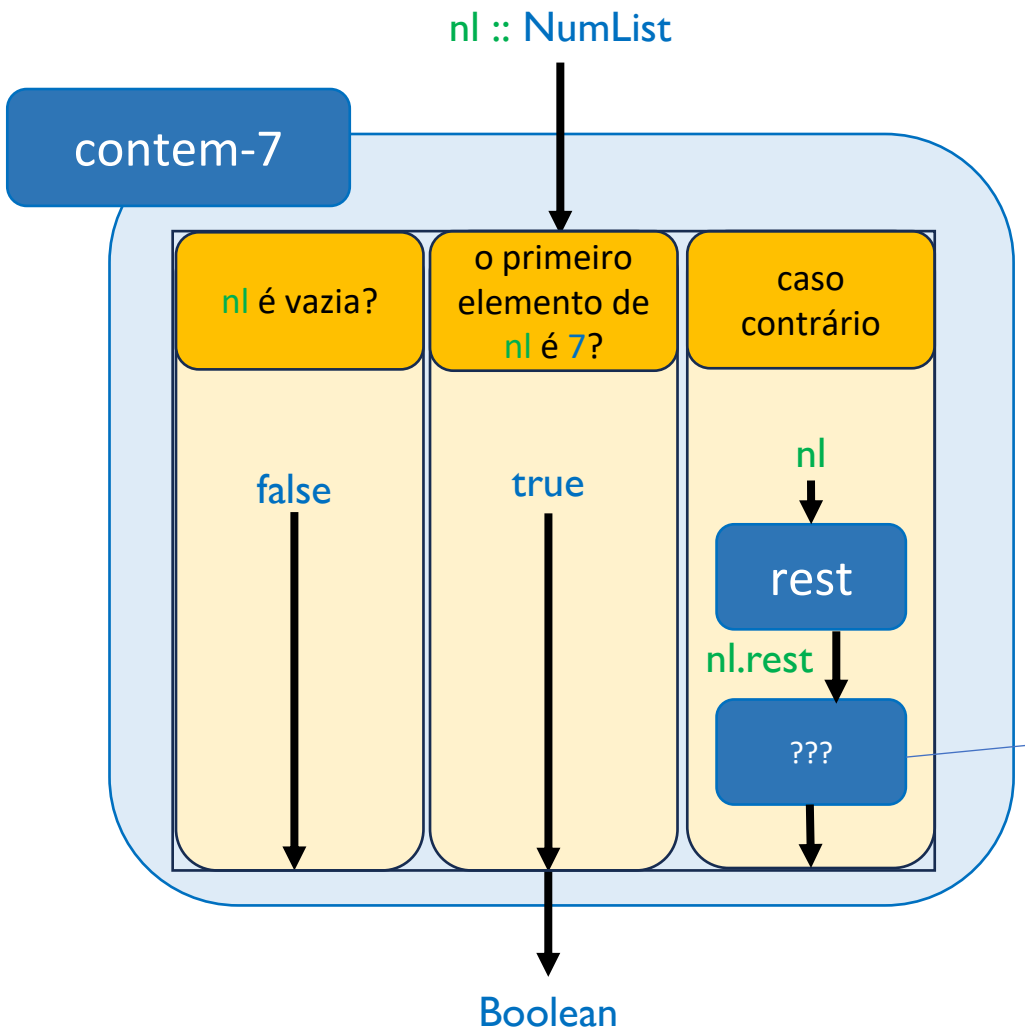

```
fun contem-7(nl :: NumList) -> Boolean:
  doc: "Dado uma lista de números, retorna true se
ela contém o número 7, e false caso contrário."

   # Se o primeiro número da lista for 7, então a
retorna true
   if nl.first == 7:
      true
   # Se não, verificar se o resto da lista contem o
número 7
   else:
       if nl.rest.first == 7:
         true
       else:
         if nl.rest.rest.first == 7:
            true
          else:
            ...
   end

where:
  contem-7(nl-empty) is false
  contem-7(nl-link(1, nl-link(7, nl-empty))) is true
  contem-7(nl-link(1, nl-link(2, nl-link(3, nl-
empty)))) is false
end
```

Quando parar?

Contem 7

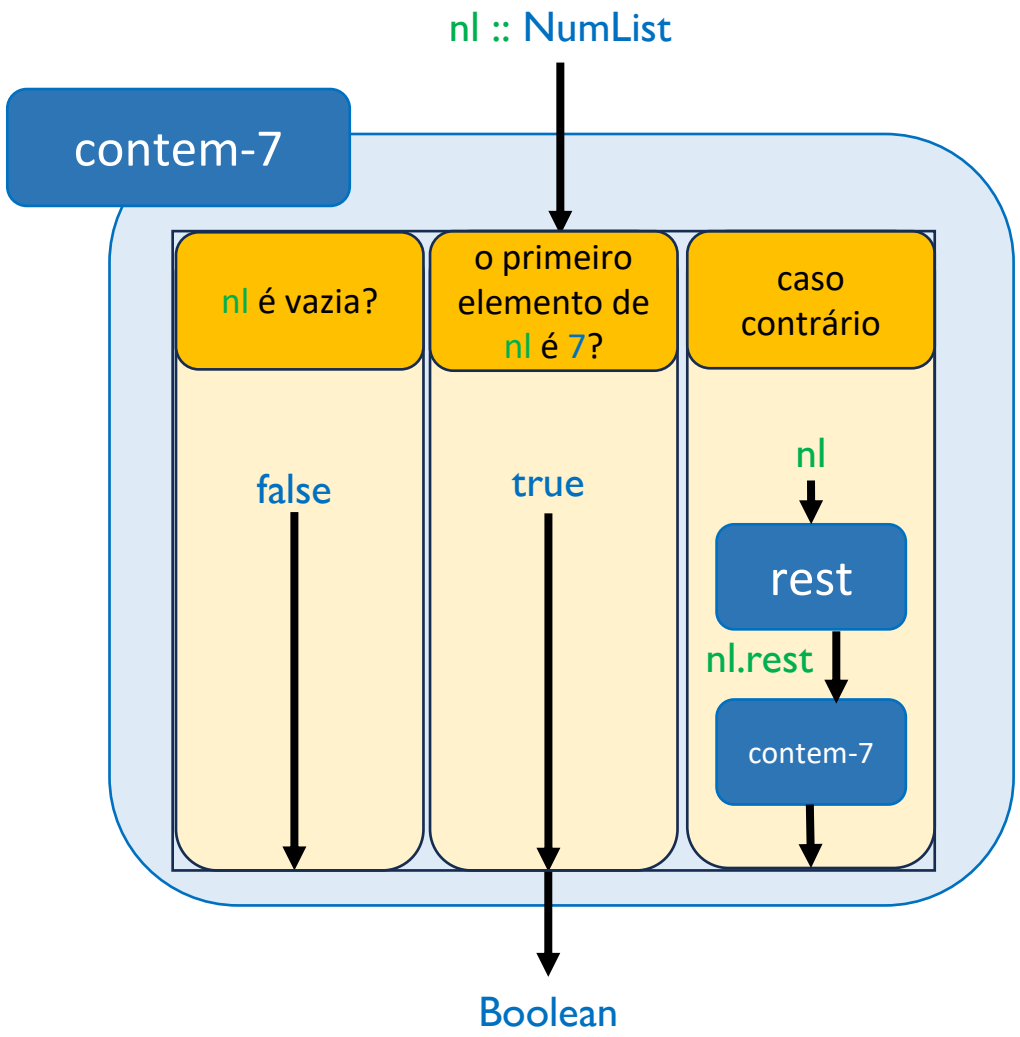



```
fun contem-7(nl :: NumList) -> Boolean:
  doc: "Dado uma lista de números, retorna true se ela
contém o número 7, e false caso contrário."
  cases (NumList) nl:
    # Se a lista for vazia, ela não contém o número 7
    | nl-empty => false

    # Se a lista não for vazia:
    | nl-link(primeiro, resto) =>
        # Se o primeiro número da lista for 7
        if primeiro == 7:
            true
        # Se não, verificar se o resto da lista contem o 7
        else:
            ???(resto)
        end
  end
where:
  contem-7(nl-empty) is false
  contem-7(nl-link(1, nl-link(7, nl-empty))) is true
  contem-7(nl-link(1, nl-link(2, nl-link(3, nl-empty)))) is
false
end
```

Parar o quê?

Contem 7




```
fun contem-7(nl :: NumList) -> Boolean:
  doc: "Dado uma lista de números, retorna true se ela
contém o número 7, e false caso contrário."
  cases (NumList) nl:
    # Se a lista for vazia, ela não contém o número 7
    | nl-empty => false

    # Se a lista não for vazia:
    | nl-link(primeiro, resto) =>
        # Se o primeiro número da lista for 7
        if primeiro == 7:
            true
        # Se não, verificar se o resto da lista contem o 7
        else:
            contem-7(resto)
        end
  end
where:
  contem-7(nl-empty) is false
  contem-7(nl-link(1, nl-link(7, nl-empty))) is true
  contem-7(nl-link(1, nl-link(2, nl-link(3, nl-empty)))) is
false
end
```

Contem 7

```
fun contem-7(nl :: NumList) -> Boolean:
  doc: "Dado uma lista de números, retorna true se ela contém o número 7, e false caso contrário."
  cases (NumList) nl:
    # Se a lista for vazia, ela não contém o número 7
    | nl-empty => false
    # Se a lista não for vazia:
    | nl-link(primeiro, resto) =>
        # Se o primeiro número da lista for 7, então a retorna true
        if primeiro == 7:
            true
        # Se não, verificar se o resto da lista contem o número 7
        else:
            contem-7(resto)
        end
  end
where:
  contem-7(nl-empty) is false
  contem-7(nl-link(1, nl-link(2, nl-link(3, nl-empty)))) is false
  contem-7(nl-link(1, nl-link(7, nl-empty))) is true
end
```

Expressão **cases**

- Listas são um tipo de dados com mais de uma variante (empty, e link).

- Em Pyret (e outras linguagens funcionais), há um tipo de expressão condicional especial para tratar cada variante: o **cases**

Expressão **cases**

Em Pyret, distinguimos as variantes de um dado condicional com a expressão **cases**:

```
cases (Tipo) e:
  | variante-1(x, y) => <expressão>
  | ...
  | variante-n(x, y, z) => <expressão>
end
```

- e é uma expressão que avalia para um valor do tipo Tipo
- Primeiro, e é avaliado e é checado se e é do tipo Tipo. Se não for, erro de execução.
- Se é do Tipo, checa qual variante, e retorna a expressão a direita do => correspondente .

Expressão **cases**

Em Pyret, distinguimos as variantes de um dado condicional com a expressão **cases**:

```
cases (NumList) nl:
   | nl-empty => <expressão>
   | nl-link(f, r) => <expressão>
end
```

- `nl` é uma expressão que avalia para um valor do tipo `NumList`
- `f` e `r` são nomes dados para os atributos first e rest
- O lado direito de todo "`=>`" é uma expressão

Dado vs. Programa

```
data NumList:
    | nl-empty

    | nl-link(
       first :: Number,
       rest :: NumList)
end
```

```
cases (NumList) nl:
   | nl-empty => <expressão>

   | nl-link(f, r) => <expressão>
end
```

Execução

Vamos simular a chamada:

contem-7(nl-link(2, nl-link(3, nl-link(7, nl-empty))))

```
fun contem-7(nl :: NumList) -> Boolean:
  cases (NumList) nl:
    # Se a lista for vazia, ela não contém o número 7
    | nl-empty => false
    # Se a lista não for vazia:
    | nl-link(primeiro, resto) =>
        # Se o primeiro número da lista for 7, então a retorna true
        if primeiro == 7:
            true
        # Se não, verificar se o resto da lista contem o número 7
        else:
            contem-7(resto)
        end
  end
end
```

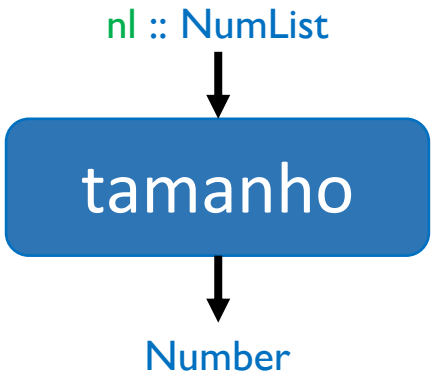
Problema

tamanho

Obj: Dada uma lista de números, retornar o seu tamanho.

Entrada: Uma lista de números (NumList).

Saída: *O tamanho da lista, um número*.



Tamanho

```
fun tamanho(nl :: NumList) -> Number:
  doc: "Dado uma lista de números, retorna seu tamanho."
  cases (NumList) nl:
    # Se a lista for vazia, ela tem tamanho 0
    | nl-empty => 0
    # Se a lista não for vazia:
    | nl-link(primeiro, resto) => 1 + tamanho(resto)
   end
where:
  tamanho(nl-empty) is 0
  tamanho(nl-link(1, nl-link(2, nl-link(3, nl-empty)))) is 3
  tamanho(nl-link(1, nl-link(7, nl-empty))) is 2
end
```

Recursão

- Uma forma abstrata de construir soluções baseada em decomposição

- Ideia é decompor (quebrar) um problema grande em problemas menores do mesmo tipo do problema original
- Então montar a solução através da combinação das soluções dos problemas menores (composição)

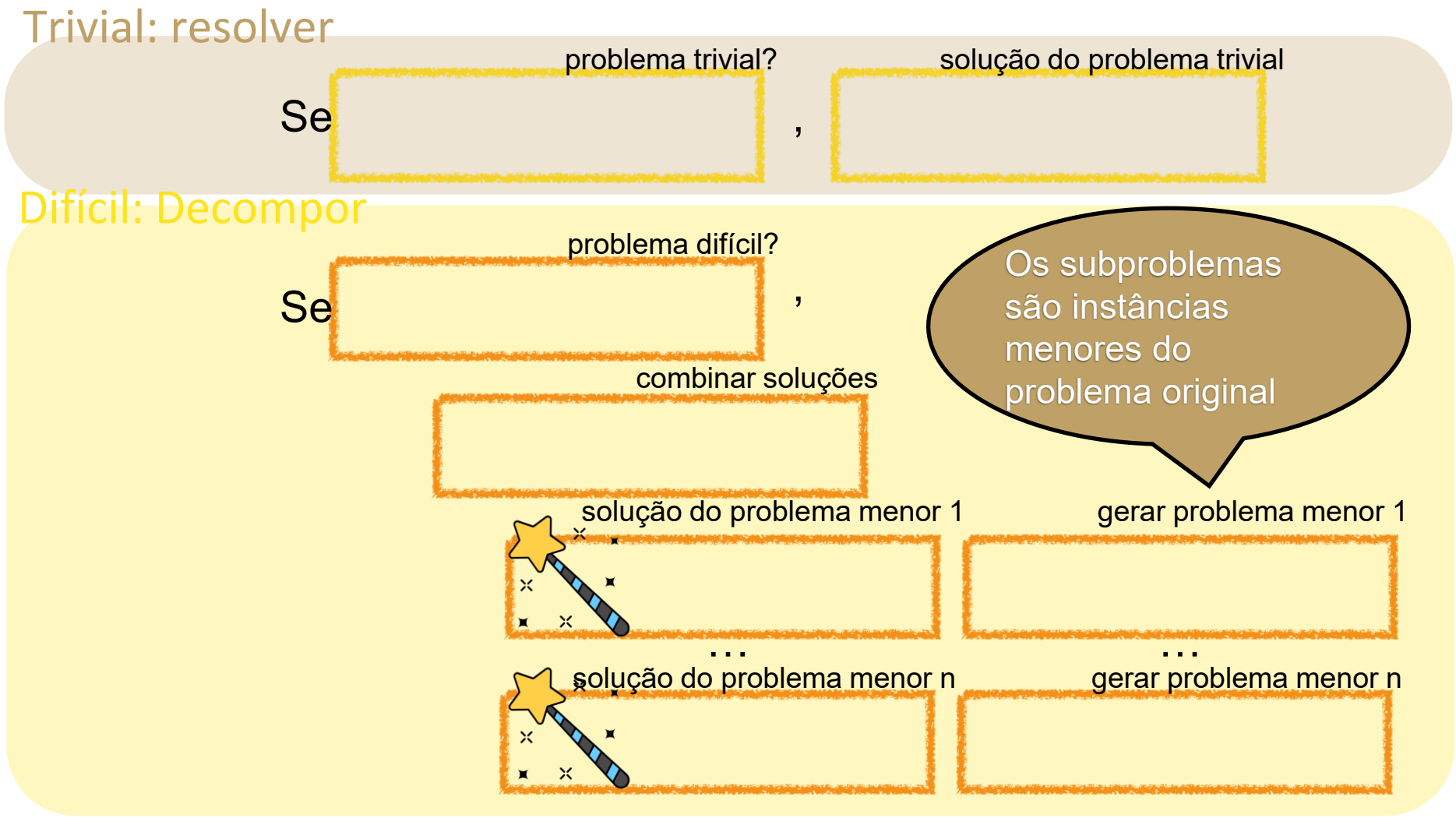
- Consideramos o caso base (fim) e o passo, **assumindo que o problema está resolvido para os problemas menores**, e pensando como **combinar** essas soluções para construir a solução do problema original.

Recursão

Ideia geral:

- Se o problema for trivial, resolva-o; (fim)
- Caso contrário:
  - Quebre/decomponha o problema
  - Assuma que os problemas menores estão solucionados
  - Combine as soluções

Modelo da solução



Template para Recursão de Listas

```
fun num-list-fun(nl :: NumList) -> ???:
  cases (NumList) nl:
    | nl-empty => ...

    | nl-link(first, rest) =>
      ... first ...
      ... num-list-fun(rest) ...
  end
end
```

- Ler: https://dcic-world.org/2025-08-27/recursive-data.html#(part._.The_.Design_.Recipe)

Recursão na Matemática

Descrição abstrata:

$$\text{factorial}(n) = \begin{cases} 1, \text{ se } n = 0 \\ n \times \text{factorial}(n-1), \text{ se } n > 0 \end{cases}$$

Execução:

$$\begin{cases} 1, \text{ se } 3 = 0 \\ 3 \times \text{factorial}(2), \text{ se } 3 > 0 \end{cases}$$

$$\begin{cases} 1, \text{ se } 2 = 0 \\ 2 \times \text{factorial}(1), \text{ se } 2 > 0 \end{cases}$$

$$\begin{cases} 1, \text{ se } 1 = 0 \\ 1 \times \text{factorial}(0), \text{ se } 1 > 0 \end{cases} = 1$$

$$\begin{cases} 1, \text{ se } 0 = 0 \\ 0 \times \text{factorial}(-1), \text{ se } 0 > 0 \end{cases} = 1$$

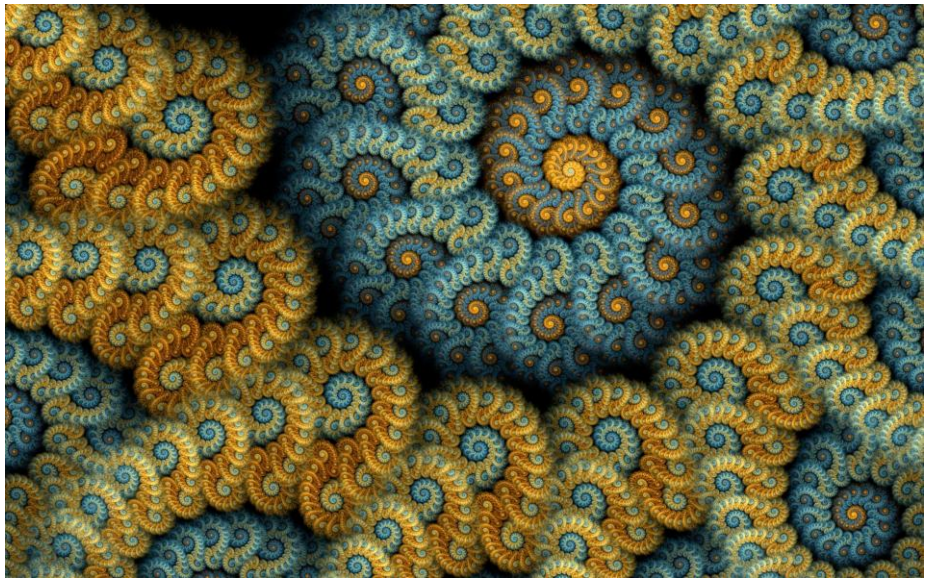
$= 2$

$= 6$

Recursão em Artes



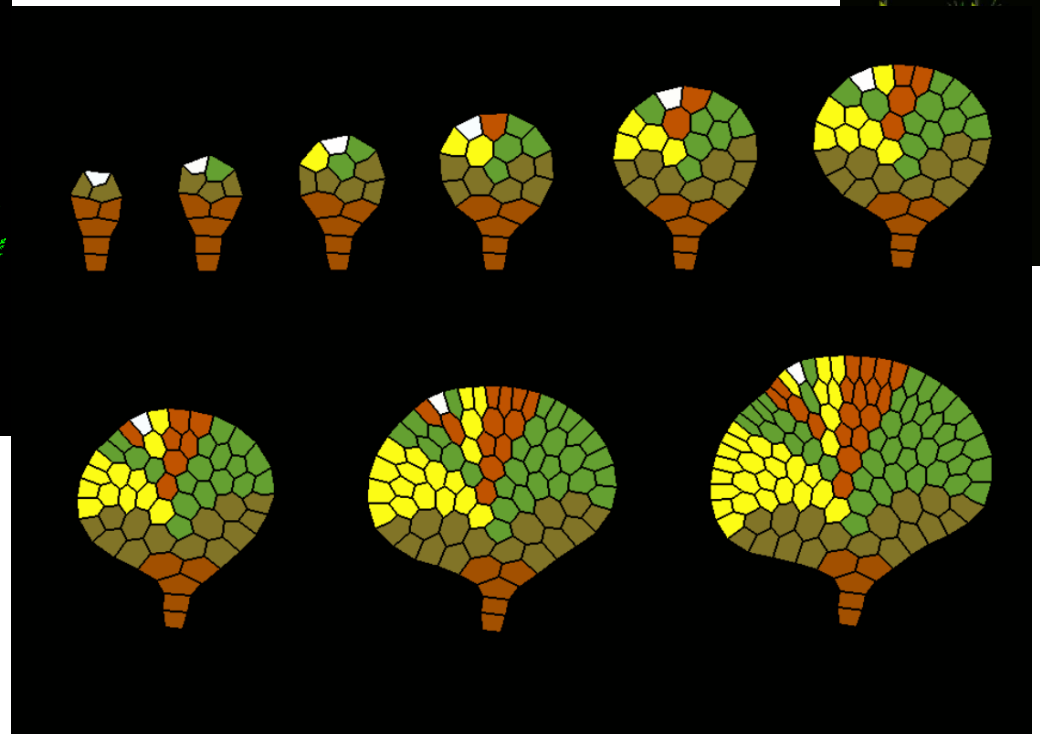


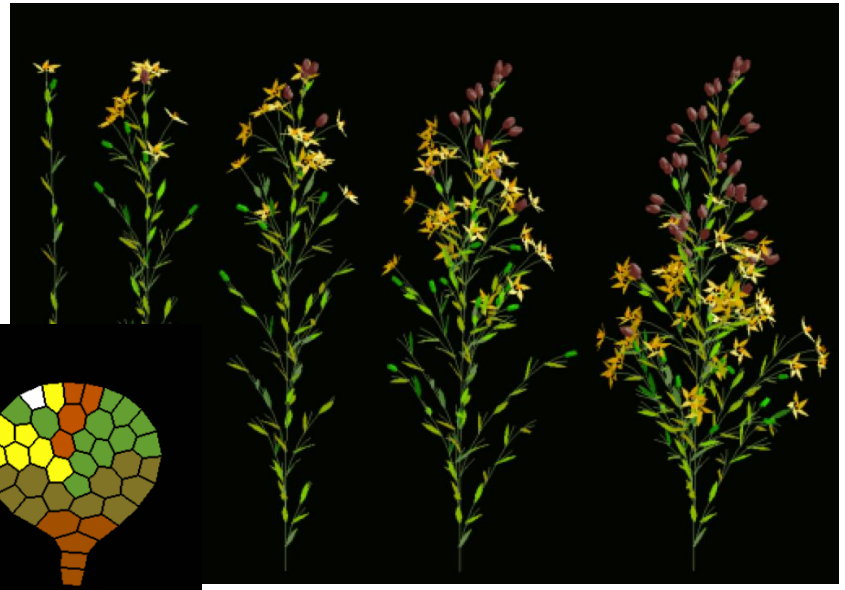




Recursão na Natureza

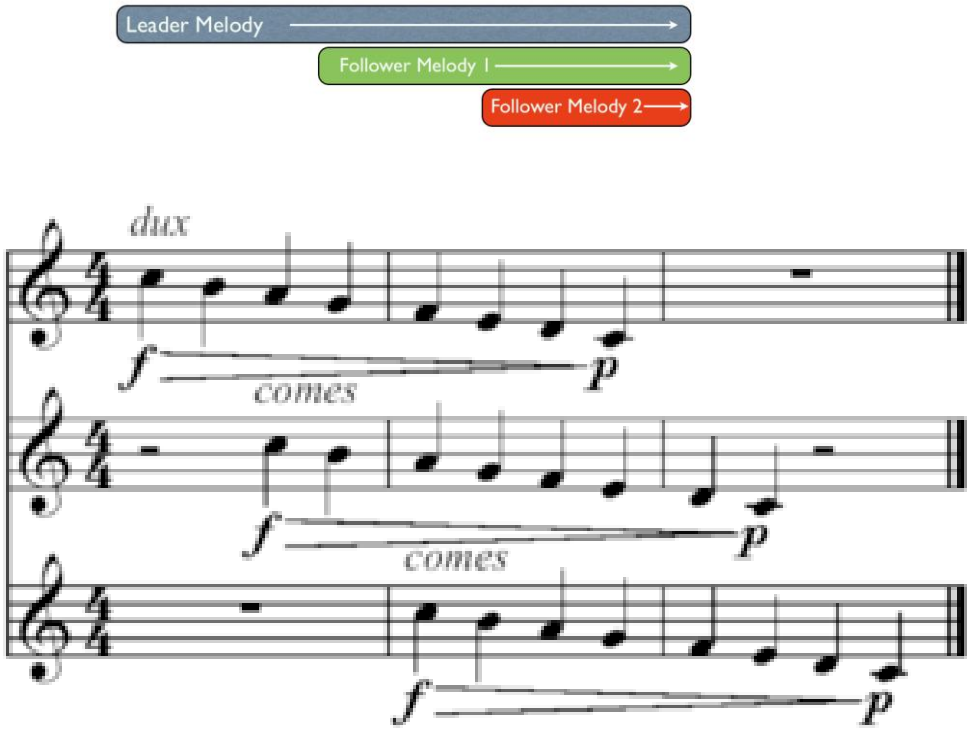




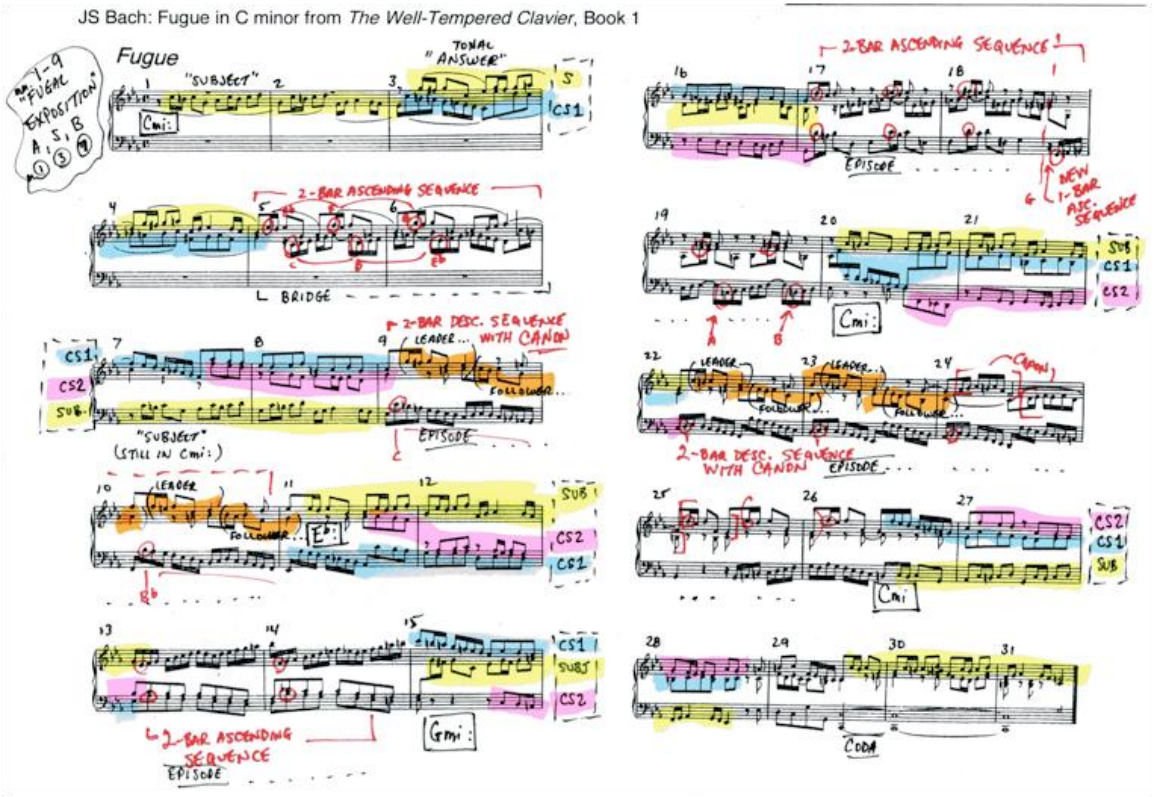


Recursão na Música

Cânones:



Fugas:



Recursão na Literatura

